

Informe de Funciones de Activación en Neuronas Artificiales

Chanco Capucho Fabrizzio Renzo
Estudiante de Ing. Sistemas
Universidad Nacional del Centro del Peru
Huancayo, Peru
e_2025100772E@uncp.edu.pe
 <https://orcid.org/0009-0008-6483-5417>

Abstract—Este informe presenta un análisis detallado de seis funciones de activación fundamentales utilizadas en el diseño de redes neuronales artificiales: ReLU, Leaky ReLU, Sigmoide, Softmax, Lineal y GELU. Se explora su formulación matemática, contexto histórico, ventajas comparativas y costo computacional. Adicionalmente, se documenta la implementación práctica de cada función mediante scripts en Python utilizando NumPy y Keras, visualizando tanto el comportamiento de la función como su integración en una arquitectura neuronal básica.

Index Terms—Redes Neuronales, Funciones de Activación, Aprendizaje Profundo, Costo Computacional, Python.

I. INTRODUCCIÓN

Las redes neuronales artificiales son el núcleo del aprendizaje profundo moderno. Sin embargo, una red neuronal compuesta únicamente por transformaciones lineales sería incapaz de resolver problemas complejos, reduciéndose a un simple modelo de regresión lineal. Las funciones de activación introducen la no linealidad necesaria para que la red pueda aprender y modelar patrones intrincados en los datos.

Este documento desglosa seis de las funciones de activación más prominentes en la ingeniería de software y la inteligencia artificial, evaluando su base teórica, su evolución histórica y su impacto computacional, respaldado por implementaciones prácticas.

II. RECTIFIED LINEAR UNIT (ReLU)

A. Concepto Matemático

La función ReLU es una transformación matemática sencilla que mantiene los valores positivos inalterados y convierte cualquier valor negativo a cero. Matemáticamente se define como:

$$f(x) = \max(0, x) \quad (1)$$

B. Historia y Popularidad

Aunque los conceptos detrás de ReLU se propusieron a finales de los años 60, su uso se masificó alrededor de 2010 gracias a investigadores como Vinod Nair y Geoffrey Hinton. Su adopción fue clave al mitigar un problema grave en las redes profundas: el desvanecimiento del gradiente (cuando las redes dejan de aprender en las capas más profundas). Frente a otras funciones más antiguas, ReLU permite que las redes se entrenen de manera más rápida y eficiente.

C. Rango de Salida y Aplicaciones

El rango de salida de esta función es de $[0, \infty)$. En la ingeniería actual, es la opción predeterminada para las capas ocultas de casi cualquier red neuronal profunda, especialmente en sistemas de visión artificial y clasificación de imágenes.

D. Ventajas y Desventajas

- **Ventaja:** Acelera drásticamente la convergencia del modelo y es computacionalmente muy eficiente.
- **Desventaja:** Sufre del problema de “neuronas muertas”; si los pesos se ajustan mal, la neurona solo recibirá valores negativos, emitiendo un cero perpetuo y dejando de aprender.

E. Costo Computacional

El costo computacional es muy bajo. Para esta función, la CPU o GPU solo necesita realizar una operación lógica simple (evaluar si un número es mayor que cero). Al no requerir cálculos de punto flotante pesados, exponenciales ni divisiones, permite procesar millones de neuronas en fracciones de segundo.

F. Implementación y Resultados

El script desarrollado genera un rango de valores y aplica la regla del máximo. Se utilizó *Keras* para validar el resultado frente a nuestra propia función de *NumPy*.

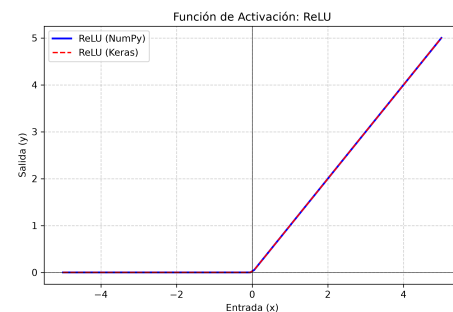


Fig. 1. Comportamiento de la función ReLU ante valores positivos y negativos.

Neurona con activación: ReLU

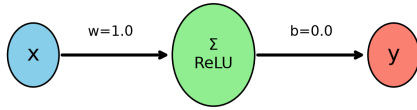


Fig. 2. Diagrama de la neurona con activación ReLU y su salida generada.

III. LEAKY ReLU

A. Concepto Matemático

Leaky ReLU es una variante de ReLU diseñada para corregir su principal limitación. En lugar de convertir los números negativos a cero absoluto, los multiplica por una constante muy pequeña (usualmente $\alpha = 0.01$).

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases} \quad (2)$$

B. Historia y Popularidad

Esta variante fue introducida poco después de la adopción masiva de ReLU para solucionar el problema de las “neuronas muertas”. En el ReLU clásico, si una neurona recibe continuamente valores negativos, su salida se estanca en cero. Leaky ReLU permite una pequeña fuga de información (de ahí el término *leaky*), lo que mantiene la neurona activa durante el entrenamiento.

C. Rango de Salida y Aplicaciones

El rango de salida abarca todos los números reales, de $(-\infty, \infty)$. Se utiliza frecuentemente en Redes Generativas Antagónicas (GANs) o como sustituto directo cuando un modelo con ReLU no logra aprender correctamente.

D. Ventajas y Desventajas

- **Ventaja:** Garantiza que el gradiente nunca sea cero, evitando la muerte prematura de las neuronas.
- **Desventaja:** Introduce un nuevo hiperparámetro (α) que el ingeniero debe ajustar manualmente durante el diseño del modelo.

E. Costo Computacional

El costo computacional se mantiene bajo. Sigue la misma lógica de comparación que ReLU, agregando únicamente una multiplicación escalar condicional (multiplicar por α cuando el valor es negativo). Estas multiplicaciones básicas son operaciones sumamente rápidas a nivel de hardware, por lo que el tiempo de procesamiento extra es casi imperceptible.

F. Implementación y Resultados

El código en Python evalúa la condición lineal, aplicando la penalización a los valores por debajo de cero, obteniendo los siguientes resultados gráficos.

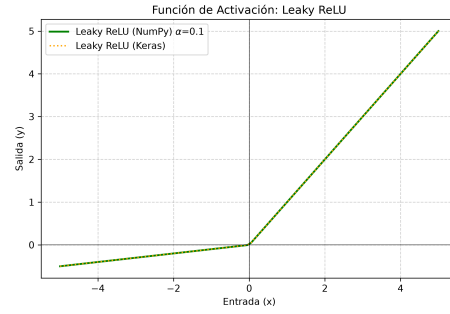


Fig. 3. Comportamiento de la función Leaky ReLU mostrando una ligera pendiente negativa.

Neurona con activación: Leaky ReLU

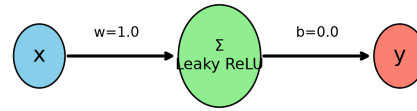


Fig. 4. Diagrama de la neurona con activación Leaky ReLU.

IV. SIGMOIDE

A. Concepto Matemático

La función Sigmoide toma cualquier número real y lo comprime en un rango estricto entre 0 y 1, creando una curva característica en forma de “S”.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

B. Historia y Popularidad

Inspirada en la estadística y la regresión logística, fue el estándar en las redes neuronales durante los años 80 y 90 debido a que sus resultados se interpretan naturalmente como probabilidades. Aunque hoy ha sido desplazada por ReLU en las capas internas, sigue siendo fundamental en la capa final para tareas de clasificación binaria.

C. Rango de Salida y Aplicaciones

Su rango de salida está estrictamente acotado a $(0, 1)$. Su aplicación principal se encuentra en la capa de salida de modelos de decisión dual; por ejemplo, sistemas médicos para determinar si un diagnóstico es positivo o negativo, o filtros de correo para decidir si un mensaje es *spam*.

D. Ventajas y Desventajas

- **Ventaja:** Normaliza cualquier entrada a un valor probabilístico directo y fácil de interpretar.
- **Desventaja:** Para valores muy altos o muy bajos, la curva se vuelve plana, deteniendo el aprendizaje del modelo (saturación de gradientes).

E. Costo Computacional

El costo computacional es alto. Matemáticamente, exige calcular el número de Euler elevado a una potencia negativa (e^{-x}), sumar un entero y finalmente realizar una división. Estas funciones exponenciales consumen una mayor cantidad de ciclos de reloj en el procesador en comparación con sumas o multiplicaciones simples.

F. Implementación y Resultados

Mediante el script, se implementó la fórmula con la librería matemática de *NumPy* y se comparó con la capa Dense de *Keras* con activación *sigmoid*.

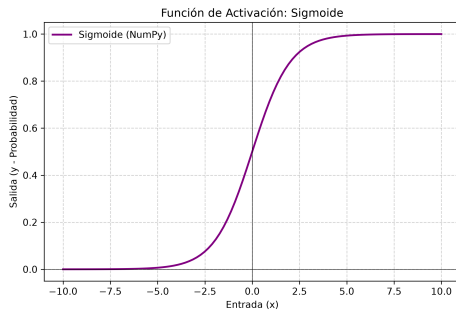


Fig. 5. Curva característica en forma de "S" de la función Sigmoide.

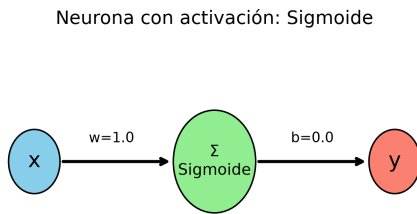


Fig. 6. Diagrama de la neurona con activación Sigmoide.

V. SOFTMAX

A. Concepto Matemático

Softmax es una generalización de la función sigmoide adaptada para múltiples dimensiones. Toma un vector de números (logits) y los normaliza en una distribución de probabilidad, donde cada valor se sitúa entre 0 y 1, y la suma total es exactamente 1, como se define a continuación:

$$S(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad (4)$$

B. Historia y Popularidad

Basada en conceptos de la mecánica estadística, Softmax es indispensable en problemas de clasificación multiclase. Si un sistema necesita evaluar una imagen y decidir si es un gato, un perro o un pájaro, Softmax garantiza que las probabilidades calculadas para todas las opciones sumen el 100%.

C. Rango de Salida y Aplicaciones

Cada valor de salida oscila entre (0,1). Es el estándar absoluto para la capa final de redes neuronales que clasifican múltiples categorías simultáneamente, como el reconocimiento de caracteres ópticos (OCR) o la catalogación automatizada de productos.

D. Ventajas y Desventajas

- **Ventaja:** Transforma valores aislados en una distribución de probabilidad mutuamente excluyente, forzando a la red a destacar la opción más probable.
- **Desventaja:** Debido a su naturaleza exponencial, es extremadamente sensible a valores atípicos (*outliers*), lo que puede polarizar la predicción si los datos de entrada no están bien escalados.

E. Costo Computacional

Es la función con el costo computacional más alto del grupo evaluado. Exige calcular una función exponencial para cada elemento del vector de salida, luego iterar para sumar todos esos resultados y, finalmente, dividir cada exponencial por la suma total. Debido a este volumen de operaciones, se utiliza casi exclusivamente en la última capa de la red y no en las capas ocultas.

F. Implementación y Resultados

En el código, se restó el valor máximo antes de la exponencial para mantener la estabilidad numérica y evitar desbordamientos de memoria.

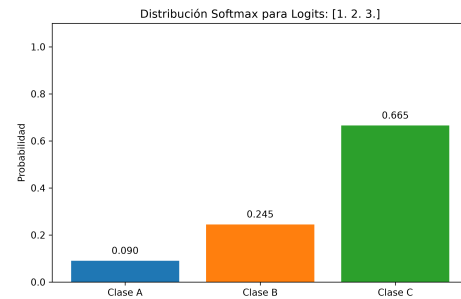


Fig. 7. Distribución probabilística generada por la función Softmax.

Neurona con activación: Softmax

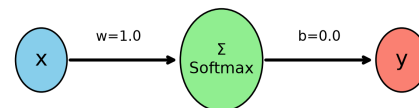


Fig. 8. Diagrama conceptual de la activación Softmax.

VI. LINEAL (IDENTIDAD)

A. Concepto Matemático

La función lineal, o de identidad, devuelve el mismo valor que recibe como entrada sin aplicar ninguna alteración.

$$f(x) = x \quad (5)$$

B. Historia y Popularidad

Aunque puede parecer contradictorio utilizar una función que no modifica los datos, es indispensable en problemas de regresión continua. Por ejemplo, al predecir magnitudes físicas o económicas, el sistema debe ser capaz de emitir cualquier número real. Usar una función diferente limitaría o distorsionaría el valor final.

C. Rango de Salida y Aplicaciones

El rango de salida abarca todos los números reales $(-\infty, \infty)$. En la práctica, es la opción estándar para la capa de salida en tareas predictivas. Un caso aplicado es la predicción de variables en sistemas de monitoreo agrícola, como la estimación precisa de la temperatura ambiente o la humedad del suelo para evaluar la viabilidad de cultivos de papa y maíz basados en sensores.

D. Ventajas y Desventajas

- **Ventaja:** Preserva la magnitud exacta de los datos proyectados sin recortes ni distorsiones.
- **Desventaja:** Si se aplica en las capas ocultas, colapsa toda la red neuronal a una simple matriz lineal, impidiendo que el modelo aprenda patrones complejos o curvas.

E. Costo Computacional

El costo computacional es virtualmente nulo. Al devolver la entrada intacta, el hardware no realiza procesamiento aritmético adicional; es simplemente una operación de paso de datos donde la salida refleja la suma calculada previamente por la neurona.

F. Implementación y Resultados

El script de prueba evidencia que las entradas generadas coinciden perfectamente de forma lineal con las salidas.

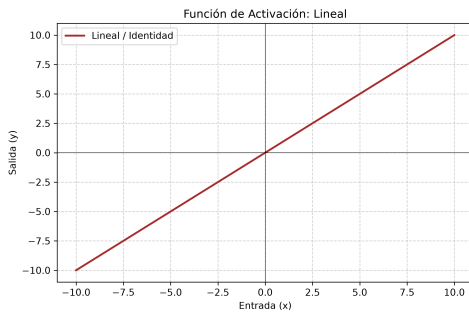


Fig. 9. Proyección directa de la función Lineal o de Identidad.

Neurona con activación: Lineal

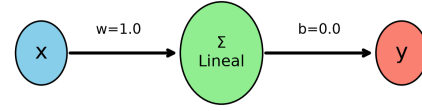


Fig. 10. Diagrama de la neurona con activación Lineal.

VII. GAUSSIAN ERROR LINEAR UNIT (GELU)

A. Concepto Matemático

GELU pondera las entradas basándose en una distribución normal (gaussiana). Para fines prácticos, se suele utilizar una aproximación matemática muy precisa:

$$f(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right) \quad (6)$$

B. Historia y Popularidad

Introducida en 2016, GELU es una función moderna que ganó notoriedad por ser el estándar en modelos de lenguaje avanzados (arquitecturas Transformer) como BERT y GPT. Combina las propiedades de ReLU con un enfoque probabilístico, lo que permite a la red tomar decisiones más equilibradas y suaves sobre la activación de las neuronas.

C. Rango de Salida y Aplicaciones

Su rango oscila aproximadamente desde $(-0.17, \infty)$. Actualmente es el pilar de los modelos de inteligencia artificial generativa y procesamiento de lenguaje natural (NLP), utilizándose intensivamente en el análisis de texto y la generación de código.

D. Ventajas y Desventajas

- **Ventaja:** Ofrece una transición probabilística suave, lo que mejora significativamente el rendimiento en redes neuronales altamente densas y complejas.
- **Desventaja:** Su alta complejidad matemática incrementa considerablemente los tiempos de entrenamiento en comparación con funciones simples como ReLU.

E. Costo Computacional

El costo computacional es considerablemente alto. Su fórmula requiere elevar números al cubo (x^3), realizar multiplicaciones con decimales extensos, calcular raíces cuadradas y aplicar una función tangente hiperbólica ($\tanh$). A pesar de esta exigencia de hardware, los modelos modernos asumen el costo a cambio del gran beneficio en el rendimiento final.

F. Implementación y Resultados

La codificación requirió el uso de `np.tanh` y constantes matemáticas precisas para evaluar el comportamiento de la curva cerca del valor cero.

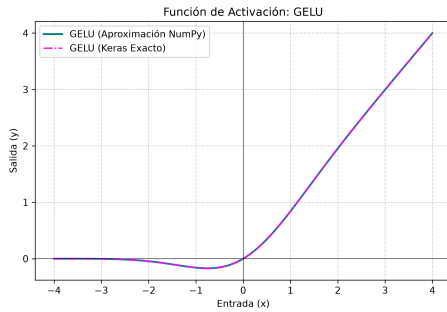


Fig. 11. Curva de activación suave característica de la función GELU.

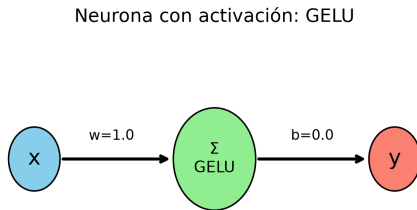


Fig. 12. Diagrama de la neurona con activación GELU.

VIII. METODOLOGÍA DE IMPLEMENTACIÓN COMPUTACIONAL

Para generar los resultados visuales y validar los conceptos matemáticos, se estructuró un entorno de pruebas en *Python* compuesto por los siguientes pasos:

- **Cálculo Numérico:** Se utilizó la librería *NumPy* para aplicar de manera directa la fórmula matemática sobre un conjunto de datos controlados.
- **Validación Práctica:** Se instanció un modelo básico en *Keras* con pesos manuales, lo que permitió confirmar que los resultados teóricos coinciden con los obtenidos por herramientas utilizadas en la industria.
- **Generación Gráfica:** Mediante *Matplotlib*, operando en segundo plano, se exportaron tanto las gráficas de comportamiento (.png) como los diagramas estructurales (_neurona.png), asegurando una presentación limpia para este informe.

IX. GLOSARIO DE TÉRMINOS

Para una lectura más fluida, se definen a continuación algunos de los conceptos técnicos mencionados:

- **Logits:** Los valores puros que una red neuronal produce antes de ser procesados por una función final como Softmax.
- **Gradiente Desvaneciente:** Fenómeno donde las señales de aprendizaje se vuelven tan débiles que las primeras partes de la red dejan de ajustarse y aprender.
- **Neurona Muerta:** Una unidad de la red que, debido a un mal ajuste previo, comienza a generar únicamente

ceros (en el caso de ReLU) y pierde la capacidad de actualizarse.

- **Capa Oculta:** Cualquier nivel de procesamiento interno de la red que se encuentra entre la entrada inicial de datos y el resultado final.
- **Clasificación Binaria / Multiclase:** Tareas de predicción; binaria implica elegir entre dos opciones (ej. encendido o apagado), mientras que multiclase evalúa tres o más posibilidades simultáneamente.

X. CONCLUSIÓN

La elección de una función de activación responde directamente al objetivo que cumple cada sección de la red neuronal. Las funciones más simples y de bajo costo, como ReLU y sus variaciones, son ideales para procesar grandes volúmenes de datos en las capas intermedias. Por el contrario, funciones probabilísticas y más exigentes a nivel de hardware, como Softmax, GELU y Sigmoide, resultan clave para tareas especializadas y la interpretación final de los resultados. Comprender cómo operan matemáticamente y cómo impactan en el rendimiento del procesador es un paso esencial para el diseño eficiente de sistemas de inteligencia artificial.